# Session 03 - Menu Loop and In-Memory Documents - Tests

Python Contoso Vault - 15 November 2026

## Part 1 - Core: vault.py

### Goal

Wrap the cipher in a menu that keeps running until the user quits. Let the user create documents, list them and read them back. Keep each body encrypted in memory, and give the reader three attempts to type the right key.

Run `python vault.py` in the terminal and type each input line, then press Enter. To try one function, open the Python shell in the same folder and run `from vault import check_key, create_document`.

### Provided

Copy your own `caesar.py` from your Session 02 folder into this folder. It defines `encrypt(plaintext: str, key: int) -> str` and `decrypt(ciphertext: str, key: int) -> str`.

The menu is these four lines:

```
1. Create
2. List
3. Read
4. Quit
```

### Instructions

1. Import `encrypt` and `decrypt` from `caesar.py`.
2. Define a function `create_document` that takes the documents as a list, a title as a string, a body as a string and a key as an integer.
   2.1. Give the document a unique ID, as an integer. IDs count up from 1 in the order the documents are created, so the third document has ID 3.
   2.2. Encrypt the body with the key.
   2.3. Encrypt `CONTOSO-VAULT` with the key, as the check value.
   2.4. Build the document as a dictionary with the keys `id`, `title`, `body` and `check`.
   2.5. Add the document to the end of the documents.
   2.6. Return the document as a dictionary.
3. Define a function `check_key` that takes a document as a dictionary and a key as an integer.
   3.1. Decrypt the document's check value with the key.
   3.2. Return `True` if the result is `CONTOSO-VAULT`, and `False` otherwise.
4. When the file runs as a program:
   4.1. Start the documents as an empty list.
   4.2. Repeat until the user quits:
      4.2.1. Print the menu.
      4.2.2. Read a choice as a string.
      4.2.3. On `1`:
         4.2.3.1. Read a title, a body and a key.
         4.2.3.2. Create the document.
         4.2.3.3. Print `Saved document N.`, where `N` is its ID.
      4.2.4. On `2`:
         4.2.4.1. Print one line `ID. Title` per document, in ID order.
         4.2.4.2. Print `No documents.` if there are none and go back to the menu.
      4.2.5. On `3`:
         4.2.5.1. Read an ID.
         4.2.5.2. Print `No such document.` and go back to the menu if no document has that ID.

4.2.5.3. Otherwise, give the user three attempts at the key. For each attempt:
    4.2.5.3.1. Read a key.
    4.2.5.3.2. Print the decrypted body and go back to the menu if the key is right.
    4.2.5.3.3. Print `Wrong key.` if the key is wrong.
4.2.5.4. Print `Too many attempts.` and go back to the menu after the third wrong key.
4.2.6. On `4`, print `Goodbye.` and end the program.
4.2.7. On any other choice, print `Invalid choice.`

## Constraints

- Show no prompt text when reading input, so the output holds only the required lines.
- The menu prints before every choice, as the exact four lines above.
- Every input is one line. Create reads the title, then the body, then the key. Read reads the ID, then each key.
- A key at create must be a whole number from 1 to 25. Any other whole number prints `Key must be from 1 to 25.` and asks for the key again.
- A key or ID that is not a whole number prints `Please enter a whole number.` and asks again. It does not use an attempt.
- Two keys that differ by 26 open the same document.
- `create_document` returns the document with the ciphertext as its body.
- `create_document` and `check_key` neither print nor read input.
- Importing the file reads no input and prints nothing.
- The program never stops with a Python error message, whatever the user types. It ends only when the user chooses `"4"`.

## Design

This is advice on how to build the code well. No test checks it, and your explain-back looks at it.

- Decide whether a guessed key is right with `check_key`.
- Keep only the ciphertext body in memory. Never store the plaintext body.
- Give every variable and function a type hint.

## Input validation

Required for every key and ID, as the constraints set out. Any title and body are valid, including empty ones.

## Test plan

### Function tests

`documents` starts as an empty list. `document_1` is the result of test 1.

| Test | Call | Expected output | Actual output | Pass |
|---|---|---|---|---|
| 1 | `create_document(documents, "Payroll", "Salaries go out on 28 November.", 3)` | `{"id": 1, "title": "Payroll", "body": "Vdodulhv jr rxw rq 28 Qryhpehu.", "check": "FRQWRVR-YDXOW"}`. `documents` holds it | | |
| 2 | Then `create_document(documents, "Kandy rent", "Kandy office rent is Rs 150,000.", 11)` | `{"id": 2, "title": "Kandy rent", "body": "Vlyoj zqqtnp cpye td Cd 150,000.", "check": "NZYEZDZ-GLFWE"}`. `documents` holds 2 | | |
| 3 | `create_document([], "Empty", "", 5)` | `{"id": 1, "title": "Empty", "body": "", "check": "HTSYTXT-AFZQY"}` | | |
| 4 | `check_key(document_1, 3)` | `True` | | |
| 5 | `check_key(document_1, 4)` | `False` | | |
| 6 | `check_key(document_1, 29)` | `True` | | |
| 7 | `check_key(document_1, -23)` | `True` | | |
| 8 | `check_key(document_1, 0)` | `False` | | |

**Program tests**

`python vault.py` reads the input lines in order. "Create Payroll" stands for the four input lines `"1"`, `"Payroll"`, `"Salaries go out on 28 November."`, `"3"`. Every run prints the menu before each choice, as usual. Every run ends with the `Goodbye.` line. The cells show only the other lines.

| Test | Input lines | Expected output | Actual output | Pass |
|---|---|---|---|---|
| 9 | `4` | No other lines | | |
| 10 | `2`, `4` | `No documents.` | | |
| 11 | Create Payroll, `2`, `4` | `Saved document 1.`, `1. Payroll` | | |
| 12 | Create Payroll, `3`, `1`, `3`, `4` | `Saved document 1.`, `Salaries go out on 28 November.` | | |
| 13 | Create Payroll, `3`, `1`, `5`, `3`, `4` | `Saved document 1.`, `Wrong key.`, `Salaries go out on 28 November.` | | |
| 14 | Create Payroll, `3`, `1`, `5`, `6`, `7`, `2`, `4` | `Saved document 1.`, `Wrong key.` three times, `Too many attempts.`, `1. Payroll` | | |
| 15 | Create Payroll, `3`, `1`, `abc`, `5`, `6`, `3`, `4` | `Saved document 1.`, `Please enter a whole number.`, `Wrong key.`, `Wrong key.`, `Salaries go out on 28 November.` | | |
| 16 | `1`, `Payroll`, `Salaries go out on 28 November.`, `abc`, `0`, `26`, `3`, `4` | `Please enter a whole number.`, `Key must be from 1 to 25.`, `Key must be from 1 to 25.`, `Saved document 1.` | | |
| 17 | `3`, `1`, `4` | `No such document.` | | |
| 18 | Create Payroll, `3`, `two`, `0`, `4` | `Saved document 1.`, `Please enter a whole number.`, `No such document.` | | |
| 19 | `7`, `x`, empty line, `4` | `Invalid choice.` three times | | |
| 20 | Create Payroll, `1`, `Kandy rent`, `Kandy office rent is Rs 150,000.`, `11`, `2`, `3`, `2`, `11`, `4` | `Saved document 1.`, `Saved document 2.`, `1. Payroll`, `2. Kandy rent`, `Kandy office rent is Rs 150,000.` | | |
| 21 | `import vault` | Reads no input and prints nothing | | |

In every program test, the program ends after `Goodbye.` and shows no Python error message.

## Done criteria

- ☐ Every test plan row gives exactly the expected result.
- ☐ The script runs from the terminal with `python vault.py`.

# Part 2 - Challenge: leak.py

## Goal

Show why the stored check value is a weakness. Recover a document's key from its check value alone, without trying any keys.

Keep `caesar.py` and `vault.py` in the same folder as `leak.py`. Open the Python shell there, run `from leak import recover_key`, then call `recover_key` with each document.

## Provided

`vault.py` defines `create_document(documents, title, body, key) -> dict` and `check_key(document, key) -> bool`. `caesar.py` defines `decrypt(ciphertext, key) -> str`. Import them only if `recover_key` uses them.

## Instructions

1. Define a function `recover_key` that takes a stored document as a dictionary, as `create_document` returns it, with the keys `id`, `title`, `body` and `check`.
   1.1. Work out the key from the document's check value, which is `CONTOSO-VAULT` encrypted with that key. The key is an integer from 1 to 25.
   1.2. Return the key as an integer.

## Constraints

- The returned key opens the document: `check_key(document, recover_key(document))` is `True`.
- The function returns its result. It neither prints nor reads input.
- Importing the file reads no input and prints nothing.

## Design

This is advice on how to build the code well. No test checks it, and your explain-back looks at it.

- Work out the key without trying keys.
- Call `decrypt` and `check_key` at most once per document.
- Give every variable and function a type hint.

## Input validation

Not required. Assume every document comes from `create_document` with a key from 1 to 25.

## Test plan

| Test | Document | Expected output | Actual output | Pass |
|---|---|---|---|---|
| 1 | Check value `FRQWRVR-YDXOW` | `3` | | |
| 2 | Check value `NZYEZDZ-GLFWE` | `11` | | |
| 3 | Check value `DPOUPTP-WBVMU` | `1` | | |
| 4 | Check value `BNMSNRN-UZTKS` | `25` | | |
| 5 | Check value `PBAGBFB-INHYG` | `13` | | |
| 6 | 30 documents from `create_document`, keys 1 to 25 | Each returns its own key | | |

## Done criteria

☐ Every test plan row returns exactly the expected key.